

P3445R0

Add utilities for easier type/bit casting in `std::simd`

Published Proposal, 2024-10-16

This version:

<http://wg21.link/P3440R5>

Author:

[Daniel Towner](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Two types of casting are very commonly found in code which uses `std::simd`: type-casts which change the element type while keeping the same number of elements, and bit-casts which can change both the type and the number of elements while keeping the underlying bit representation the same. Both of these are verbose when used and would benefit from alternative forms which make them more concise and more useable.

Table of Contents

1 Motivation

1.1 Type casting

1.2 Bit-casting

2 Implementation experience

3 Wording

3.1 Add [**simd.casts**] to the synopsis

3.2 Add new simd cast section [**simd.casts**]

§ 1. Motivation

There are two types of casting in `std::simd` - type-casting and bit-casting - which would benefit from alternative forms which make them simpler to use, more readable, and provide some extra utility which is otherwise unavailable.

§ 1.1. Type casting

Type casting in `std::simd` occurs when the programmer wants to change the type (and therefore value) of the elements of a `basic_simd` without changing the number of elements. The constructors in `std::simd` allow this to be expressed by a direct call to a constructor, or through a `static_cast`, but the type definition of the destination must be setup in advance (e.g., using `rebind_simd_t`). This can lead to code which is verbose, which in turn makes it difficult the code difficult to read and understand quickly. We suggest that a short-hand called `simd_cast` is provided to make it easy to create the new type and perform the element-by-element cast. The following code shows two ways to write a function using existing facilities:

```
template<typename T, typename ABI>
auto incrementAsFloat1(const basic_simd<T, ABI>& x)
{
    // Use constructor.
    return rebind_simd_t<float, basic_simd<T, ABI>>(x) + 1.0f;
}

template<typename T, typename ABI>
auto incrementAsFloat(const basic_simd<T, ABI>& x)
{
    // Use static_cast
    using OUT = simd<float, basic_simd<T, ABI>::size>;
    return static_cast<OUT>(x) + 1.0f;
}
```

Note that there are other ways to write these, perhaps using extra aliases or intermediate temporary variables to improve readability. However, it is much cleaner to be able to write them like this instead:

```
template<typename T, typename ABI>
auto incrementAsFloat(const basic_simd<T, ABI>& x)
{
    return simd_cast<float>(x) + 1.0f;
}
```

The `simd_cast` variant is much more readable and it captures the intent of the programmer very concisely.

Of course there are times when the type-conversion may also require a change in ABI (e.g., to create a new element type with an alternative target type) in which `simd_cast` wouldn't suffice, but for the majority of the code we have encountered the simpler `simd_cast` captures the programmer's requirements.

Another good reason to use `simd_cast` is that it enables simd-generic code to be written. It is desirable to be able to write a generic algorithm once, initially using scalar types to get the code working, and then later substituting with a `simd` type. In such code we want a uniform way to cast from the implementation value type to a new type without having to reflect upon whether the type is simd or scalar. A variant of `simd_cast` can be provided which works on scalar types or simd types. For example, the following code could work with either in that case:

```
// simd-generic function
auto incrementAsFloat(auto x) {
    return simd_cast<float>(x) + 1.0f;
}

// Call with scalar:
auto w1 = incrementAsFloat(23.f);

// Call with simd:
auto w2 = incrementAsFloat(simd<int>(ptr));
```

Finally, it is worth noting that while it would be straight-forward for programmers to define `simd_cast` or an equivalent themselves, it is such a common utility that it is better for it to be defined just once in `std::simd` itself.

§ 1.2. Bit-casting

The second type of casting operation is bit-casting, where the underlying bit pattern is interpreted as though it were a different `std::simd` value. In such a conversion not only can the element type change, but the number of elements could also change. For example, a `simd<uint16_t, 8>` could be bit-cast into `simd<uint8_t, 16>` to access the individual bytes of the original.

The existing `std::bit_cast` function already allows a value of a `basic_simd` type to be bit-cast into a different `basic_simd` type:

```
// Do something to a complex simd value ([R2663R5]).
template<typename T, typename ABI>
auto fn(const basic_simd<std::complex<T>, ABI>& x)
{
    // Setup a type to represent the raw floating point elements in the
    // complex simd types. This doubles the number of elements in the simd.
    constexpr int numNativeCmplxElements = simd<std::complex<float>>::size;
    using AsFloat = simd<float, numNativeCmplxElements * 2>;

    // Do the bit-cast conversion to obtain the raw float elements.
    auto asT = std::bit_cast<AsFloat>(x);
    auto result = ...; // e.g., call an Intel intrinsic like _mm512_fmsubadd_

    // Convert back to its original complex form.
    return std::bit_cast<basic_simd<std::complex<T>, ABI>>(result);
}
```

This example shows the verbose mechanics of making that conversion. The correct type needs to be created by calculating the appropriate number of new elements for the bit-cast element type, and then creating a suitable type using those elements, before `std::bit_cast` can be called. It would be more convenient to be able to specify the new element type as part of a new `simd_bit_cast` function:

```
template<typename T, typename ABI>
auto fn(const basic_simd<std::complex<T>, ABI>& x)
{
    auto asT = simd_bit_cast<T>(x);
    auto result = ...; // e.g., call an Intel intrinsic like _mm512_fmsubadd_

    return simd_bit_cast<std::complex<T>>(result);
}
```

The `simd_bit_cast` can also be overloaded for scalar types to allow simd-generic programming. It will be equivalent to a `std::bit_cast` in those circumstances.

Like `simd_cast`, the `simd_bit_cast` is easily defined by the programmer if they want it, but it is such a useful function that it should be defined once in `std::simd` for all programmers to use.

Note that no functions will be provided to bit-cast `simd_mask` types because the underlying implementation of a mask can vary by target.

§ 2. Implementation experience

In Intel's implementation of `std::simd` the `simd_bit_cast` function was added very early on because it is so widely used.

The implementation of Intel's `std::simd` itself uses `simd_bit_cast` to make it easier to interface to compiler intrinsics. Intrinsics often require particular data types to be used to achieve certain effects, and the bit-cast allows the underlying bits to be quickly and easily reinterpreted.

Intel uses `std::simd` in a number of internal software projects, and some of those (particularly wireless or packet-processing) need to be able to easily reinterpret the underlying bits in different ways.

§ 3. Wording

§ 3.1. Add [**simd.casts**] to the synopsis

Add the following to the [**simd.syn**] section:

```
// [simd.copy], basic_simd cast functions

template<typename To, typename From> constexpr To simd_cast(const From& x)

template<typename To, typename From, typename Abi>
constexpr rebind_simd_t<To, basic_simd<From, Abi>>
simd_cast(& x);

template<typename To, typename From> constexpr To simd_bit_cast(const From& x)

template<typename To, typename From, typename Abi>
constexpr simd<To, (basic_simd<From, Abi>::size * sizeof(From)) / sizeof(From)>
simd_bit_cast(const basic_simd<From, Abi>& x);
```

§ 3.2. Add new simd cast section [**simd.casts**]

◆ **basic_simd** casts [**simd.casts**]

- 1) `template<typename To, typename From> constexpr To simd_cast(const From& x);`
- 2) `template<typename To, typename From, typename Abi>
constexpr rebind_simd_t<To, basic_simd<From, Abi>>
simd_cast(& x);`

Returns:

- For the first overload, equivalent to returning `static_cast<To>(x)`.
- For the second overload, equivalent to returning `static_cast<rebind_simd_t<To, basic_simd<From, Abi>>>(x)`.

- 1) `template<typename To, typename From> constexpr To simd_bit_cast(const From& x);`
- 2) `template<typename To, typename From, typename Abi>
constexpr simd<To, (basic_simd<From, Abi>::size * sizeof(From)) / sizeof(To)>
simd_bit_cast(const basic_simd<From, Abi>& x);`

Returns:

- For the first overload, equivalent to returning `std::bit_cast<To>(x)`.
- For the second overload, equivalent to returning `std::bit_cast<simd<To, (basic_simd<From, Abi>::size * sizeof(From)) / sizeof(To)>>(x)`.